

FastAPI

What is FastAPI?

FastAPI হলো একটা **modern, high-performance Python web framework** যেটা দিয়ে খুব সহজে এবং খুব দ্রুত **REST API** বানানো যায়।

 এটা বানানো হয়েছে:

- Speed 
- Less code 
- Automatic documentation 
- Type safety 

FastAPI কেন এত Fast?

FastAPI তৈরি হয়েছে ৩টা core জিনিসের উপর 

1. Starlette (ASGI Framework)

- Async / Await support
- Non-blocking I/O
- High concurrency

2. Pydantic

- Data validation
- Type checking
- Serialization / Deserialization

3. Python Type Hints

```
def get_user(id: int):
```

এই type hint দিয়েই FastAPI:

- validation করে
- docs বানায়
- error detect করে

⚙️ FastAPI Architecture (Inside View)

Client (Browser / App)

|

| HTTP Request

↓

FastAPI

└─ Routing (path, method)

└─ Dependency Injection

└─ Validation (Pydantic)

└─ Business Logic

|

↓

Database / External API

|

↓

JSON Response

🌱 Basic FastAPI App (Hello World)

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
```

```
def home():
```

```
    return {"message": "Hello FastAPI"}
```

▶ Run:

```
uvicorn main:app --reload
```

📄 Automatic API Documentation (Game Changer 🔥)

FastAPI নিজে নিজেই docs বানায় 🤖

- Swagger UI 👉 <http://127.0.0.1:8000/docs>
- ReDoc 👉 <http://127.0.0.1:8000/redoc>

❌ Django / Flask এ manual

✅ FastAPI তে automatic

🔄 REST API CRUD with FastAPI

Example: User API

Pydantic Model (Schema)

```
from pydantic import BaseModel
```

```
class User(BaseModel):
```

```
    name: str
```

```
    email: str
```

```
    age: int
```

+ POST (Create User)

```
@app.post("/users")
```

```
def create_user(user: User):
```

```
    return {
```

```
        "message": "User created",
```

```
        "user": user
```

```
    }
```

 **এখানে** FastAPI automatically:

- JSON → Python object
- Validation
- Error handling

GET (Read User)

```
@app.get("/users/{user_id}")

def get_user(user_id: int):

    return {"user_id": user_id}
```

PUT (Update)

```
@app.put("/users/{user_id}")

def update_user(user_id: int, user: User):

    return {"user_id": user_id, "updated": user}
```

DELETE

```
@app.delete("/users/{user_id}")

def delete_user(user_id: int):

    return {"message": "User deleted"}
```

Validation & Error Handling (Very Powerful)

 Request:

```
{

    "name": "Arman",
```

```
"email": "arman@gmail.com"
}
```

✅ Error Response (Auto):

```
{
  "detail": [
    {
      "loc": ["body", "age"],
      "msg": "field required",
      "type": "value_error.missing"
    }
  ]
}
```

👉 তুমি কিছুই লেখো নাই, FastAPI নিজেই করলো 🤖

⚡ **Async vs Sync (Huge Advantage)**

❌ **Normal (Blocking)**

```
@app.get("/sync")
def sync_api():
    time.sleep(5)
    return {"msg": "done"}
```

✅ Async (Non-Blocking)

```
@app.get("/async")
```

```
async def async_api():
```

```
    await asyncio.sleep(5)
```

```
    return {"msg": "done"}
```

📌 Async দিয়ে:

- 1 server → হাজার request handle
- High traffic support

🔑 Authentication (JWT Example Flow)

User Login

↓

JWT Token Generate

↓

Client stores token

↓

Authorization: Bearer token

FastAPI uses **Dependency Injection** 🙌

```
from fastapi import Depends
```

🌱 Dependency Injection (DI)

```
def get_db():  
    return "DB Connection"  
  
@app.get("/items")  
def read_items(db=Depends(get_db)):  
    return {"db": db}
```

🧠 Benefit:

- Clean code
- Testable
- Scalable

📖 FastAPI + Database

Popular Combos

Database	Library
PostgreSQL	SQLAlchemy / asyncpg
MongoDB	Motor
MySQL	SQLAlchemy
SQLite	SQLAlchemy

FastAPI vs Django vs Flask

Feature	FastAPI	Django	Flask
Speed	 Fastest	Medium	Medium
Async	 Native	 Partial	
Docs	 Auto		
Learning	Easy	Hard	Easy
Best For	API	Full stack	Small apps

Where FastAPI is Used?

-  Microservices
-  High-traffic APIs
-  AI / ML Model Serving
-  Real-time apps
-  FAANG backend systems

Interview One-Liner

FastAPI is a modern Python web framework built on Starlette and Pydantic that provides high performance, async support, automatic validation, and API documentation.

FastAPI হলো বর্তমানে পাইথনের অন্যতম জনপ্রিয় এবং দ্রুতগতির একটি ওয়েব ফ্রেমওয়ার্ক, যা মূলত REST API তৈরির জন্য ব্যবহৃত হয়। এটি মূলত এর গতির জন্য পরিচিত এবং এটি আধুনিক পাইথনের টাইপ হিন্টিং (Type Hinting) ব্যবহার করে তৈরি।

নিচে FastAPI-এর বিস্তারিত তথ্য দেওয়া হলো:

১. FastAPI কেন এত জনপ্রিয়?

FastAPI-এর জনপ্রিয়তার প্রধান কারণগুলো হলো:

- **High Performance:** এটি **Node.js** এবং **Go**-এর সমতুল্য গতি প্রদান করে। এর মূলে রয়েছে **Starlette** এবং **Pydantic**।
- **Fast to Code:** এটি দিয়ে খুব দ্রুত কোড লেখা যায় (প্রায় ২০০% থেকে ৩০০% দ্রুত)।
- **Automatic Documentation:** কোড লেখার সাথে সাথেই এটি অটোমেটিক **Swagger UI** এবং **ReDoc** জেনারেট করে, যার ফলে আলাদাভাবে এপিআই ডকুমেন্টেশন লিখতে হয় না।
- **Fewer Bugs:** পাইথনের টাইপ হিন্টিং ব্যবহার করার কারণে কোডে ভুলের পরিমাণ অনেক কমে যায়।

২. মূল টেকনোলজি (Core Components)

FastAPI মূলত দুটি বড় লাইব্রেরির ওপর ভিত্তি করে দাঁড়িয়ে আছে:

1. **Starlette:** এটি একটি লাইটওয়েট ASGI ফ্রেমওয়ার্ক যা ওয়েব পার্টের কাজগুলো (Routing, Request, Response) সামলায়।
2. **Pydantic:** এটি ডেটা ভ্যালিডেশনের কাজ করে। আপনি কী ধরনের ডেটা সার্ভারে পাঠাচ্ছেন তা সঠিক কি না, তা Pydantic নিশ্চিত করে।

৩. একটি ছোট কোড উদাহরণ

FastAPI ব্যবহার করে কত সহজে একটি API বানানো যায়, তা নিচের উদাহরণে দেখুন:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"Hello": "World"}

@app.get("/items/{item_id}")
def read_item(item_id: int, q: str = None):
    return {"item_id": item_id, "query": q}
```

- এখানে `@app.get("/")` হলো একটি **Route** বা পথ।
- ফাংশনের ভেতরে পাইথন টাইপ (`item_id: int`) ব্যবহার করায় FastAPI অটোমেটিক ডেটা ভ্যালিডেট করে। কেউ যদি `int`-এর বদলে স্ট্রিং পাঠায়, তবে এটি অটোমেটিক এরর মেসেজ দেবে।

৪. প্রধান বৈশিষ্ট্যসমূহ (Detailed Features)

- **Asynchronous Support (async/await):** এটি নোটিভলি `async` এবং `await` সাপোর্ট করে, যার ফলে ইনপুট/আউটপুট বা ডেটাবেস কলের সময় সার্ভার বসে থাকে না।
- **Dependency Injection:** এতে বিল্ট-ইন ডিপেন্ডেন্সি ইনজেকশন সিস্টেম আছে, যা অথেনটিকেশন বা ডেটাবেস কানেকশন হ্যান্ডেল করা সহজ করে তোলে।
- **Security and Authentication:** এতে OAuth2, JWT tokens এবং API keys সহজে ব্যবহারের সুবিধা রয়েছে।

৫. কখন FastAPI ব্যবহার করবেন?

- যখন আপনার খুব **দ্রুতগতির (High Performance)** এপিআই প্রয়োজন।

- যখন আপনার মাইক্রোসার্ভিস (Microservices) আর্কিটেকচার নিয়ে কাজ করতে হবে।
- ডেটা সায়েন্স বা মেশিন লার্নিং মডেলকে এপিআই হিসেবে এক্সপোজ করতে চাইলে।

৬. Flask vs FastAPI (পার্থক্য)

বৈশিষ্ট্য	Flask	FastAPI
গতি (Speed)	তুলনামূলক কম।	অত্যন্ত বেশি (Async সমর্থিত)।
ডেটা ভ্যালিডেশন	ম্যানুয়ালি করতে হয়।	অটোমেটিক (Pydantic দিয়ে)।
ডকুমেন্টেশন	আলাদা প্লাগইন লাগে।	বিল্ট-ইন (অটোমেটিক)।
টাইপ হিন্টিং	ব্যবহার বাধ্যতামূলক নয়।	এর ওপর ভিত্তি করেই তৈরি।